Enterprise IAM Platform — Complete Interview Study Guide

TABLE OF CONTENTS

1. Project Overview
2. What It Does (Features)
3. Architecture — How It All Flows
4. How We Built It — Step by Step in Order
5. Code Walkthrough — Key Parts Explained
6. Key Security Concepts Explained Simply
7. Real-World Use Cases
8. How to Run the Project
9. Interview Preparation — Q&A
10. Quick Revision Cheat-Sheet

SECTION 1 — PROJECT OVERVIEW

What is it? (Simple Version)

This is a web application that controls **who gets access to what** inside a company — and makes sure that access is always appropriate, approved, tracked, and taken back when no longer needed.

It is called an **Identity and Access Management (IAM)** platform.

Real-World Analogy

Think of a large office building.

- Every employee has an **access card** (their identity)
- Different doors open based on **who you are** — HR staff can enter HR rooms, Finance staff enter Finance rooms, not everyone can go everywhere
- When a **new employee joins**, they are given a card with the right doors enabled
- When someone **moves to a different department**, their card is updated — old doors removed, new ones added

- When someone **leaves the company**, their card is immediately deactivated
- If someone tries to get access to a door they shouldn't, the **system blocks them**
- Every single door entry and exit is **logged** for security review later

That is exactly what this platform does — but for **software systems** instead of physical doors.

Who Uses IAM Systems?

- **Banks** — to ensure only the right people can approve transactions
- **Hospitals** — to ensure only doctors and nurses access patient records
- **Tech companies** — to control who can deploy code, access databases, or read sensitive files
- **Government agencies** — to enforce strict separation of duties and audit trails

Why This Project Matters

In any company with more than a few employees, managing access manually (via email requests and spreadsheets) becomes a **security risk**. People leave but their accounts stay active. Someone gets too many permissions and can commit fraud. Nobody knows who approved what and when.

This platform automates and enforces all of that.

SECTION 2 — WHAT IT DOES (FEATURES)

Identity & Access Control

- **Role-Based Access Control (RBAC)** — two layers: system roles (Admin, Manager, Employee) AND resource roles (HR, Finance, IT, Security). Each user gets only what they need.
- **Least Privilege** — users start with no access. Access must be explicitly requested and approved.
- **Data Classification** — each resource role is tagged: Restricted, Confidential, Internal, or Public. Restricted roles require extra justification.
- **Time-Bound Access** — when granting a role, you can set an expiry date. After that date, access automatically shows as expired and can be bulk-revoked.

Joiner–Mover–Leaver (JML) Lifecycle

- **Joiner** — when a new employee joins, the admin creates their account and assigns starter roles in one workflow. SoD is checked even at this stage.
- **Mover** — when someone changes department or gets promoted, old roles are removed and new ones added in a single transaction. SoD is rechecked on the resulting set.
- **Leaver** — when someone leaves, one click: account disabled, all roles revoked, all pending access requests cancelled. All in one atomic operation.

Approval Workflow

- **Two-Stage Approval** — employee submits request -> manager approves/rejects -> admin does final approval/rejection -> role is provisioned. Each stage is tracked with timestamps, decisions, and comments.
- **Business Justification** — every access request requires a minimum 30-character written reason. Live character counter in the UI.

Security & Compliance

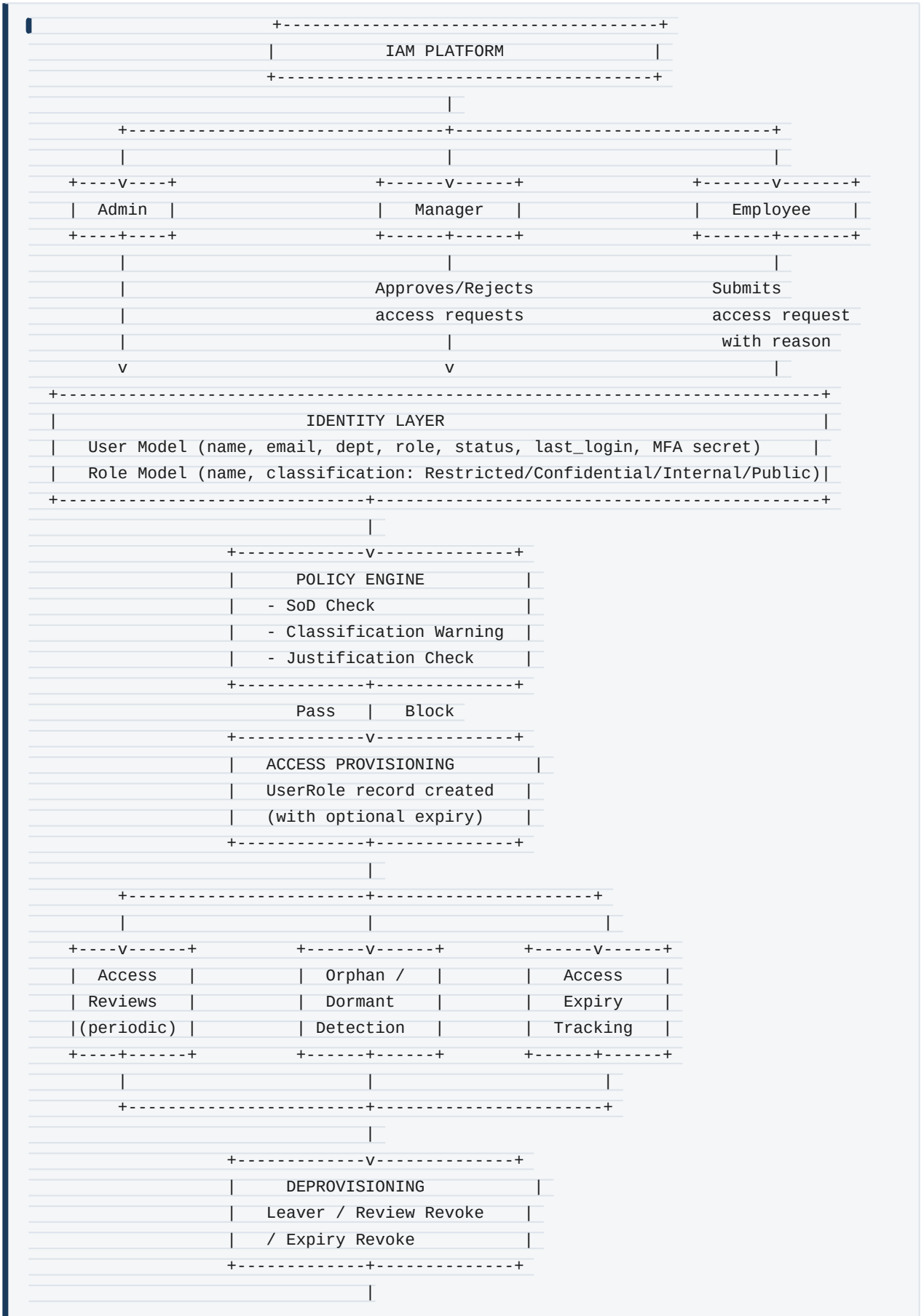
- **Segregation of Duties (SoD)** — configurable rules block incompatible role combinations. Example: Finance + Security are blocked together (to prevent fraud concealment). Finance + HR are blocked (to prevent payroll manipulation). These are checked before any role grant.
- **Policy Engine** — a centralised `policy.py` file runs before every single role assignment. If SoD is violated, the assignment is blocked. No role can be granted without passing this check.
- **Access Recertification** — quarterly (or any named period) review cycles where admins go through every active role assignment and decide: Keep or Revoke. Each decision is logged.
- **Orphan Account Detection** — detects inactive users who still hold active roles. This is a real security risk (a departed employee's account could still be used).
- **Dormant Account Detection** — detects active users who have not logged in for 90+ days. These are suspicious and should be reviewed.
- **Immutable Audit Log** — every action (login, role grant, role revoke, approval, rejection, MFA event, policy violation) is written to an append-only log with actor, timestamp, and detail. Never deleted.
- **CSV Exports** — audit log, user-role matrix, orphan accounts, review history — all exportable for compliance reports.

Authentication Technologies

- **Password Hashing** — bcrypt with salt. Passwords are never stored in plain text.
- **TOTP / MFA** — time-based one-time passwords using the pyotp library. Compatible with Google Authenticator and Authy. Per-user setup with QR code scan. Admin can force-disable if a user loses their device.
- **OAuth 2.0** — Authorization Code Flow with Google as the Identity Provider (OpenID Connect). CSRF-protected with a random state token.
- **HTTPS / TLS** — self-signed certificate generated by `generate_cert.py` using the `cryptography` library. The app auto-detects the certificate at startup and enables HTTPS.

SECTION 3 — ARCHITECTURE — HOW IT ALL FLOWS

The Big Picture (Text Diagram)



```
                    +-------------v--------------+
                    |      AUDIT LOG               |
                    |  Every action recorded.    |
                    |  Actor + Timestamp +       |
                    |  Action + Detail           |
                    +----------------------------+
```

Flow Explained Stage by Stage

Stage 1 — Identity Setup Before anything, users are created with a name, email, department, system role (Admin/Manager/Employee), and status. Resource roles (HR, Finance, IT, Security) are defined with a classification level.

Stage 2 — Access Request An employee logs in and submits a request for a resource (e.g. Finance access). They must provide a reason (at least 30 characters). The request enters the approval queue.

Stage 3 — Two-Stage Approval The request goes to the employee's manager first. Manager approves or rejects with comments. If approved, it goes to an admin for final approval. Each decision is timestamped and logged.

Stage 4 — Policy Check Before any role is provisioned, the policy engine runs. It checks SoD rules (does this new role conflict with any existing role the user has?). If a violation is found, the provisioning is blocked and logged as SOD_VIOLATION_BLOCKED.

Stage 5 — Provisioning If policy passes, a UserRole record is created linking the user to the role, with who assigned it and optionally when it expires.

Stage 6 — Ongoing Checks Access reviews run periodically — admins certify every active role assignment (Keep or Revoke). Orphan detection finds inactive users still holding roles. Dormant detection finds active users who haven't logged in for 90+ days. Expiry tracking shows roles that have passed their end date.

Stage 7 — Deprovisioning Access is taken away via: Leaver offboarding (all at once), review revocation, expiry revocation, or admin manual revoke.

Stage 8 — Audit Log Every single action throughout the entire flow is written to the audit log. The log is never modified or deleted. It can be exported to CSV for compliance reporting.

SECTION 4 — HOW WE BUILT IT — STEP BY STEP IN ORDER

Here is the exact order the project was built, and why each piece comes when it does.

STEP 1 — Database Models and Identity Foundation

What: We built the User, Role, UserRole, AccessRequest, AccessReview, AuditLog, and SodRule models in `models.py`.

How: Used Flask-SQLAlchemy (Python ORM) to define database tables as Python classes. Each column maps to a database field.

Why first: Everything else depends on the data model. You can't build login, approval workflows, or SoD checks without first defining what a User, a Role, and an Assignment look like. This is always Step 1 in any real project too.

STEP 2 — Authentication (Login, Sessions, Password Hashing)

What: Built the login/logout system in `auth.py` with bcrypt password hashing and Flask-Login session management.

How: Flask-Login manages the session (who is currently logged in). bcrypt hashes passwords before storing them. The login route checks the submitted password against the stored hash.

Why second: You cannot build role-based access control without first knowing who is logged in. Authentication (proving who you are) always comes before authorisation (deciding what you can do).

STEP 3 — Role-Based Access Control (RBAC)

What: Role-aware navigation and route protection using the `role_required()` decorator. Each route is tagged with which system role can access it.

How: A Python decorator wraps route functions. If the logged-in user's system_role doesn't match the allowed roles, they are redirected with an error.

Why third: Once we know who is logged in, we can enforce what they're allowed to see and do. RBAC is the core of access control.

STEP 4 — JML Lifecycle (Joiner, Mover, Leaver)

What: Three workflows in `routes/jml.py` to onboard, transfer, and offboard employees.

How: Joiner creates a user + assigns initial roles. Mover updates department/roles in a transaction. Leaver disables account + revokes all roles + cancels pending requests atomically.

Why fourth: Once we have users and roles, we need to manage the lifecycle. This is where "who gets what" is first established (Joiner) and finally removed (Leaver).

STEP 5 — Two-Stage Approval Workflow

What: The access request flow — employee submits -> manager reviews -> admin approves -> role provisioned.

How: The AccessRequest model is a state machine. The `stage` field tracks where in the pipeline it is: manager_review -> admin_review -> provisioned (or rejected at any stage).

Why fifth: With users and roles defined, the natural next question is "how does someone get a new role?" The answer is a controlled, auditable approval process.

STEP 6 — Segregation of Duties (SoD) and Policy Engine

What: `policy.py` with `check_sod()`, `check_sod_batch()`, and `check_policy()`. SodRule model in `models.py`. SoD management UI for admins.

How: Before any role is granted (via direct assignment, approval, Joiner, or Mover), `check_policy()` is called. It queries SodRule for conflicts. If a conflict is found, the assignment is blocked.

Why sixth: SoD is a *constraint on* provisioning. It had to come after provisioning was built, because it wraps around and validates it. We couldn't add this guard until we had something to guard.

STEP 7 — Data Classification and Business Justification

What: Added `classification` field to roles (Restricted/Confidential/Internal/Public). Added 30-character minimum justification check on access requests. Live character counter in the UI.

How: Classification is stored on the Role model. Policy engine checks it. Justification length is validated in the route before the request is saved.

Why seventh: These are refinements to the access request process — adding more intelligence to what was already built. Classification informs how sensitive a role is; justification ensures people think before requesting.

STEP 8 — Access Reviews and Recertification

What: Review cycles in `routes/reviews.py`. Admin can start a named cycle (e.g. Q3-2026), go through every active role assignment, and decide Keep or Revoke.

How: The cycle stores decisions in the AccessReview table. Revoking within a review sets the UserRole status to Revoked and writes an audit log entry.

Why eighth: Access reviews are a *continuous* compliance check on access that was already granted. They had to come after provisioning because there's nothing to review until roles are assigned.

STEP 9 — Orphan and Dormant Account Detection

What: Orphan detection (inactive users still holding active roles) and dormant detection (active users, no login 90+ days) in `routes/reviews.py`.

How: SQL queries using JOINs and date comparisons. Orphan = `User.status == Inactive AND UserRole.status == Active`. Dormant = `User.status == Active AND last_login < today - 90 days`.

Why ninth: These are detective controls. They find problems that slipped through the other processes (e.g. someone was disabled in HR but their system roles weren't revoked). They need to come after the main workflows so they have real data to detect anomalies in.

STEP 10 — MFA (TOTP) and OAuth 2.0

What: TOTP two-factor authentication (Google Authenticator compatible) and OAuth 2.0 Authorization Code Flow with Google.

How: pyotp library generates and verifies TOTP codes. After correct password, if TOTP is enabled, user is redirected to a code entry page. OAuth uses `secrets.token_urlsafe()` for CSRF state, redirects to Google, exchanges the code for a token, fetches user info, matches to local account.

Why tenth: Authentication enhancements come last because they layer on top of a working login system. The core platform needs to work first, then we strengthen the front door.

STEP 11 — Audit Logging and Governance Reporting

What: Immutable AuditLog model, `write_audit_log()` function called at every significant action, audit log viewer, governance dashboard, and CSV exports.

How: Every route that changes data calls `write_audit_log()` with actor, action type, target, and detail. Reports page queries and aggregates this data.

Why last: Audit logging wraps around everything. Each feature writes its own log entries, so the logger grows naturally as features are added. You cannot add logging before you have features to log.

SECTION 5 — CODE WALKTHROUGH

5.1 — The User Model (models.py)

```
class User(UserMixin, db.Model):
    __tablename__ = "users"

    id            = db.Column(db.Integer, primary_key=True)
    name          = db.Column(db.String(100), nullable=False)
    email         = db.Column(db.String(150), unique=True, nullable=False)
    password_hash = db.Column(db.String(255), nullable=False)
    department    = db.Column(db.String(100))
    system_role   = db.Column(db.String(20), nullable=False, default="Employee")
    status        = db.Column(db.String(20), nullable=False, default="Active")
    manager_id    = db.Column(db.Integer, db.ForeignKey("users.id"), nullable=True)
    last_login    = db.Column(db.DateTime, nullable=True)
    totp_secret   = db.Column(db.String(32), nullable=True)
    totp_enabled = db.Column(db.Boolean, default=False, nullable=False)

    def set_password(self, password: str):
        self.password_hash = bcrypt.hashpw(
            password.encode("utf-8"), bcrypt.gensalt()
        ).decode("utf-8")

    def check_password(self, password: str) -> bool:
        return bcrypt.checkpw(
            password.encode("utf-8"), self.password_hash.encode("utf-8")
        )
```

What it does: Defines what a user record looks like in the database. Each variable is a database column.

Key design choices: - `password_hash` — we never store the actual password, only the bcrypt hash. Even if the database is stolen, passwords cannot be recovered. - `manager_id` self-referencing the same table — a user can be their own manager's manager. This is called a self-referential foreign key. - `totp_secret` and `totp_enabled` — added when we implemented MFA. The secret is a random base32 string used to generate TOTP codes. - `last_login` — used for dormant account detection (no login in 90+ days). - `status` — Active or Inactive. Inactive users cannot log in, even if they know the password.

Why this matters in an interview: Shows you understand data modelling, password security best practices, and that you thought ahead about what data you'd need for each feature.

5.2 — The SoD Rule Model (models.py)

```
class SodRule(db.Model):
    __tablename__ = "sod_rules"

    id          = db.Column(db.Integer, primary_key=True)
    role_id_1   = db.Column(db.Integer, db.ForeignKey("roles.id"), nullable=False)
    role_id_2   = db.Column(db.Integer, db.ForeignKey("roles.id"), nullable=False)
    description = db.Column(db.String(200), nullable=False)
    created_by  = db.Column(db.Integer, db.ForeignKey("users.id"), nullable=True)
    created_at  = db.Column(db.DateTime, default=datetime.utcnow)

    role_1   = db.relationship("Role", foreign_keys=[role_id_1])
    role_2   = db.relationship("Role", foreign_keys=[role_id_2])
    creator  = db.relationship("User", foreign_keys=[created_by])
```

What it does: Each row represents one SoD rule — two roles that cannot be held by the same person at the same time.

Key design: The rule is bidirectional — Finance + Security is the same conflict as Security + Finance. The check_sod() function handles this by querying both directions at once.

Why this matters in an interview: Demonstrates understanding of SoD as a data-driven, configurable control — not hardcoded logic. Admins can add new rules without touching code.

5.3 — The Policy Engine (policy.py)

```
def check_sod(user_id, role_id, extra_role_ids=None):
    active_role_ids = {
        ur.role_id for ur in UserRole.query.filter_by(
            user_id=user_id, status="Active"
        ).all()
    }

    rules = SodRule.query.filter(
        db.or_(
            db.and_(SodRule.role_id_1 == role_id,
                    SodRule.role_id_2.in_(active_role_ids)),
            db.and_(SodRule.role_id_2 == role_id,
                    SodRule.role_id_1.in_(active_role_ids)),
        )
    ).all()

    violations = []
    for rule in rules:
        conflicting_id = (rule.role_id_2 if rule.role_id_1 == role_id
                          else rule.role_id_1)
        new_role       = db.session.get(Role, role_id)
        conflicting    = db.session.get(Role, conflicting_id)
        violations.append(
            f"SoD Conflict: '{new_role.name}' cannot be combined "
            f"with '{conflicting.name}' — {rule.description}"
        )
    return violations


def check_policy(user_id, role_id, justification=None):
    violations = check_sod(user_id, role_id)
    warnings   = []

    role  = db.session.get(Role, role_id)
    level = CLASSIFICATION_LEVELS.get(role.classification or "Internal", 1)
    if level >= 3:  # Restricted
        if not justification or len(justification.strip()) < 30:
            warnings.append(
                f"'{role.name}' is Restricted. "
                f"Ensure justification is detailed before provisioning."
            )

    return len(violations) == 0, violations, warnings
```

What it does: `check_sod()` gets the user's current active roles, then queries SodRule for any rule where the new role appears alongside any existing role. Returns a list of violation strings (empty list = no violation).

`check_policy()` is the master gate — it runs SoD check AND checks classification. Returns a tuple: (passed, violations, warnings).

Why bidirectional query: A rule says "Finance and Security conflict." If we only check `role_id_1 == new_role`, we'd miss cases where the new role is stored as `role_id_2`. The `db.or_()` catches both.

Why this matters in an interview: Shows you understand centralised policy enforcement. Any route that assigns a role calls this one function — consistent, single point of truth, easy to extend.

5.4 — Approval Workflow State Machine (models.py)

```
class AccessRequest(db.Model):
    # State machine: Pending -> Manager Approved -> Admin Approved
    #                        -> Provisioned (or Rejected at any stage)
    status = db.Column(db.String(30), default="Pending")
    stage  = db.Column(db.String(30), default="manager_review")
    # manager_review -> admin_review -> provisioned / rejected

    manager_decision  = db.Column(db.String(20), nullable=True)
    manager_comments  = db.Column(db.Text)
    manager_action_at = db.Column(db.DateTime, nullable=True)

    admin_decision  = db.Column(db.String(20), nullable=True)
    admin_comments  = db.Column(db.Text)
    admin_action_at = db.Column(db.DateTime, nullable=True)

    provisioned_at = db.Column(db.DateTime, nullable=True)
    expires_at     = db.Column(db.DateTime, nullable=True)
```

What it does: Every access request tracks its current stage, all decisions with timestamps, and who made each decision. This is a **state machine** — the request moves through defined states in order.

Why this matters in an interview: A state machine is a core design pattern. You can explain: "The request has two gates — manager and admin — and can only move forward through them in order. It can be rejected at either gate. I designed it this way to enforce the two-stage approval process and make the full history auditable."

5.5 — JML Leaver (routes/jml.py)

```
@jml_bp.route("/leaver/<int:user_id>", methods=["GET", "POST"])
@role_required("Admin")
def leaver(user_id):
    user             = db.session.get(User, user_id)
    active_roles     = UserRole.query.filter_by(user_id=user_id, status="Active").all()
    pending_requests = AccessRequest.query.filter(
        AccessRequest.requester_id == user_id,
        AccessRequest.stage.in_(["manager_review", "admin_review"]),
    ).all()

    if request.method == "POST":
        # 1. Disable the account
        user.status = "Inactive"

        # 2. Revoke all active role assignments
        for ur in active_roles:
            ur.status = "Revoked"

        # 3. Cancel all pending access requests
        for req in pending_requests:
            req.status = "Rejected"
            req.stage  = "rejected"
            req.admin_comments = f"Auto-cancelled: {user.name} has been offboarded."

        db.session.commit()
        write_audit_log(actor_id=current_user.id, action="JML_LEAVER", ...)
```

What it does: In one single database transaction: marks user Inactive, revokes every active role, and auto-rejects all pending requests. Everything happens atomically — either all happens or nothing does.

Why atomic: If the account is disabled but roles aren't revoked (partial failure), the account is still a risk. The transaction ensures it's all-or-nothing.

Why this matters in an interview: This is a perfect example of thinking about edge cases. "What if someone has a pending request when they leave?" We handle it — the request gets auto-cancelled with a clear reason in the comments.

5.6 — Orphan and Dormant Detection (routes/reviews.py)

```
DORMANT_DAYS = 90

# Orphan: Inactive user who still has Active roles
orphan_users = (
    User.query
    .join(UserRole, User.id == UserRole.user_id)
    .filter(User.status == "Inactive", UserRole.status == "Active")
    .distinct()
    .all()
)

# Dormant: Active user, no login in 90+ days
cutoff = datetime.utcnow() - timedelta(days=DORMANT_DAYS)
dormant_users = User.query.filter(
    User.status == "Active",
    db.or_(User.last_login == None, User.last_login < cutoff),
).all()
```

What it does: Orphan query uses a JOIN across two tables — it finds users who are marked Inactive but still have at least one UserRole with status Active. Dormant query finds users who are Active but either never logged in OR last logged in more than 90 days ago.

Why this matters in an interview: These are detective controls. They find security gaps that slipped through the main processes. Real companies run these reports daily or weekly. The 90-day threshold is a common industry standard.

5.7 — TOTP / MFA Implementation (auth.py)

```
# During login — if user has TOTP enabled, intercept:
if user.totp_enabled:
    session["pending_totp_user_id"] = user.id
    return redirect(url_for("auth.totp_verify"))

# TOTP verification route:
@auth_bp.route("/totp/verify", methods=["GET", "POST"])
def totp_verify():
    user_id = session.get("pending_totp_user_id")
    user    = db.session.get(User, user_id)

    code = request.form.get("code", "").strip()
    totp = pyotp.TOTP(user.totp_secret)

    if totp.verify(code, valid_window=1):   # allows ±30s clock drift
        session.pop("pending_totp_user_id", None)
        _complete_login(user, method="totp")
        return redirect(url_for("auth.dashboard"))
    else:
        write_audit_log(..., action="MFA_FAILED", ...)
        flash("Invalid code. Please try again.", "danger")

# TOTP setup — generate QR code:
secret = pyotp.random_base32()
provisioning_uri = pyotp.TOTP(secret).provisioning_uri(
    name=current_user.email,
    issuer_name="IAM Lab",
)
# URI is rendered as QR code in the browser using qrcodejs
```

What it does: After correct password, if TOTP is enabled, the user's ID is stored in a temporary session variable and they're redirected to the code entry page (they're not logged in yet). The TOTP code is verified against their stored secret. `valid_window=1` allows one 30-second window of tolerance for clock differences between the server and phone.

Setup flow: A random base32 secret is generated, stored temporarily in the session, and displayed as a QR code. The user scans it with their authenticator app. Only after they submit a valid code is the secret saved to their account.

Why this matters in an interview: You can explain what TOTP is: "TOTP generates a 6-digit code using the shared secret and the current time. Both the server and the authenticator app do the same calculation independently and should get the same number. If they match, authentication succeeds."

5.8 — OAuth 2.0 Authorization Code Flow (auth.py)

```
@auth_bp.route("/oauth/google")
def oauth_google():
    # Step 1: Generate CSRF state token
    state = secrets.token_urlsafe(16)
    session["oauth_state"] = state

    params = {
        "client_id":     client_id,
        "redirect_uri":  redirect_uri,
        "response_type": "code",
        "scope":         "openid email profile",
        "state":         state,
    }
    # Redirect user to Google
    auth_url = GOOGLE_AUTH_URL + "?" + urllib.parse.urlencode(params)
    return redirect(auth_url)


@auth_bp.route("/oauth/google/callback")
def oauth_callback():
    # Step 2: Verify CSRF state
    if request.args.get("state") != session.pop("oauth_state", None):
        flash("OAuth security check failed.", "danger")
        return redirect(url_for("auth.login"))

    # Step 3: Exchange authorization code for access token (server-to-server)
    code = request.args.get("code")
    token_resp = requests.post(GOOGLE_TOKEN_URL, data={
        "code":          code,
        "client_id":     client_id,
        "client_secret": client_secret,
        "redirect_uri":  redirect_uri,
        "grant_type":    "authorization_code",
    })
    access_token = token_resp.json().get("access_token")

    # Step 4: Get user info from Google
    userinfo = requests.get(
        GOOGLE_USERINFO_URL,
        headers={"Authorization": f"Bearer {access_token}"}
    ).json()
    google_email = userinfo.get("email")

    # Step 5: Match to local IAM account
    user = User.query.filter_by(email=google_email).first()
    if not user:
        flash("No IAM account found for this Google email.", "danger")
        return redirect(url_for("auth.login"))
```

```
    _complete_login(user, method="oauth")
```

What it does: Implements the OAuth 2.0 Authorization Code Flow — the most secure OAuth flow.

The flow in plain English: 1. User clicks "Continue with Google" 2. We generate a random state token (CSRF protection) and redirect to Google 3. Google authenticates the user and redirects back to our callback URL with an authorization code 4. We verify the state token matches (prevents CSRF attacks) 5. We exchange the code for an access token (this is server-to-server — the client secret never leaves our server) 6. We use the access token to get the user's email from Google 7. We look up that email in our local database 8. If found and active, we log them in

Why Authorization Code Flow: The access token is never exposed to the user's browser. It's exchanged server-to-server. This is more secure than the Implicit Flow (older, deprecated) where the token was directly in the browser URL.

Why this matters in an interview: This is a frequent question in security-focused roles. You can walk through each step and explain why each one exists (especially the state token for CSRF and why the code-to-token exchange is server-side).

5.9 — Audit Logging (models.py)

```
class AuditLog(db.Model):
    __tablename__ = "audit_logs"

    id          = db.Column(db.Integer, primary_key=True)
    actor_id    = db.Column(db.Integer, db.ForeignKey("users.id"), nullable=True)
    action      = db.Column(db.String(50), nullable=False)
    target_type = db.Column(db.String(30))    # User, AccessRequest, Role, System
    target_id   = db.Column(db.Integer, nullable=True)
    details     = db.Column(db.Text)
    timestamp   = db.Column(db.DateTime, default=datetime.utcnow)
    ip_address  = db.Column(db.String(45))

def write_audit_log(actor_id, action, target_type=None, target_id=None,
                    details=None, ip_address=None):
    entry = AuditLog(
        actor_id=actor_id, action=action,
        target_type=target_type, target_id=target_id,
        details=details, ip_address=ip_address,
    )
    db.session.add(entry)
    db.session.commit()
```

What it does: Every significant action in the system calls `write_audit_log()`. The log records who did what, to what, when, and from which IP address. The log is never updated or deleted — only new

entries are added.

Key logged events include: LOGIN_SUCCESS, LOGIN_FAILED, LOGIN_BLOCKED, LOGOUT, ROLE_ASSIGNED, ROLE_REVOKED, USER_CREATED, USER_DEACTIVATED, JML_JOINER, JML_MOVER, JML_LEAVER, REQUEST_SUBMITTED, REQUEST_APPROVED, REQUEST_REJECTED, REVIEW_CYCLE_STARTED, REVIEW_COMPLETED, REVIEW_CYCLE_CLOSED, SOD_VIOLATION_BLOCKED, ORPHAN_REMEDIATED, ACCESS_EXPIRED, MFA_ENABLED, MFA_DISABLED, MFA_FAILED, MFA_ADMIN_DISABLED, OAUTH_NO_ACCOUNT

Why immutable: Audit logs are a legal and compliance requirement. If they can be edited, they lose their value as evidence. In production, you'd write to a write-once storage system or a SIEM.

Why this matters in an interview: Auditing is fundamental to ISO 27001 and most security frameworks. Being able to say "every action generates an immutable log entry with actor, timestamp, and detail" is exactly what security teams want to hear.

SECTION 6 — KEY SECURITY CONCEPTS EXPLAINED SIMPLY

IAM — Identity and Access Management

What it is: The practice of ensuring the right people have the right access to the right resources at the right time.

Simple analogy: A hotel key card system. Each guest gets a card that opens their room, the pool, and the gym — but not other guests' rooms or staff areas. When they check out, the card stops working.

RBAC — Role-Based Access Control

What it is: Instead of assigning permissions directly to individual people, you create roles (like "Finance Analyst") and assign permissions to the role. Then you assign people to roles.

Why: If 50 people are Finance Analysts, you don't set up permissions 50 times. You set them up once on the role. When the Finance rules change, you update the role and everyone gets updated automatically.

In our project: Two-layer RBAC. System roles (Admin/Manager/Employee) control what pages you see. Resource roles (HR/Finance/IT/Security) control which company data you can access.

Least Privilege

What it is: Every person gets only the minimum access they need to do their job — nothing more.

Why: If a user with excessive permissions is compromised (hacked, or goes rogue), the damage they can cause is limited to what they had access to.

In our project: Users start with no resource roles. They must request access with a business justification, go through approval, and pass policy checks before getting it.

Joiner–Mover–Leaver (JML)

What it is: A framework for managing access throughout the full employee lifecycle.

- **Joiner:** New employee starts. Account created, starter roles assigned, manager assigned.
- **Mover:** Employee changes job or department. Old roles removed, new roles granted. Must re-pass SoD checks.
- **Leaver:** Employee resigns or is dismissed. Account disabled immediately, all roles revoked, all pending requests cancelled.

Why it matters: Most access security failures happen at Leaver. Someone leaves a company but their account stays active for weeks or months. That's how ex-employees steal data.

Segregation of Duties (SoD)

What it is: No single person should be able to complete a high-risk process from start to finish without a check.

Classic example: The same person should not be able to both create a supplier invoice AND approve the payment to that supplier. If they could, they could pay themselves or a fake company.

In our project: - Finance + Security are blocked (someone who can see financial data AND security systems could cover their tracks if they steal money) - Finance + HR are blocked (someone who can process payroll AND manage employee records could add fake employees to the payroll)

How it works: The SodRule table stores these pairs. The policy engine checks before every role grant.

Access Reviews / Recertification

What it is: Periodically (quarterly, annually), managers and admins go through a list of all active access assignments and confirm: "Yes, this person still needs this access" or "No, revoke it."

Why it matters: People's roles change. They get promoted, change teams, or change responsibilities — but old access is rarely taken away proactively. Access reviews catch this drift.

In our project: Admins start a named cycle (e.g. "Q4-2026"), go through every active UserRole, and decide Keep or Revoke. All decisions are logged.

Orphan Accounts

What it is: A user account that is disabled (the person left) but the access permissions were never removed.

Real risk: If someone knows the credentials of an orphan account, or if the account can be reactivated, they could use it to access systems under a former employee's identity.

In our project: The orphan detection query finds any user with `status = Inactive` who still has a UserRole with `status = Active`. The admin can remediate (revoke all orphan roles) with one click.

Dormant Accounts

What it is: A user account that is still active but has not been used for a long time (90+ days in our system).

Real risk: A dormant account could be a former employee who hasn't been properly offboarded, a service account that's no longer needed, or an account that's been forgotten and is at risk of being taken over.

In our project: Detected by checking `last_login < today - 90 days OR last_login IS NULL`. Admins can disable the account directly from the detection report.

MFA / TOTP

What it is: Multi-Factor Authentication adds a second check on top of the password. TOTP (Time-Based One-Time Password) generates a 6-digit code that changes every 30 seconds.

How TOTP works: 1. During setup, the server generates a random secret key and shows it as a QR code 2. The user scans it with Google Authenticator — the app now has the shared secret 3. At login, both the server and the app use the same formula: HMAC(secret + current_time_in_30s_intervals) 4. They should produce the same 6-digit number. If they match, the code is valid.

Why it matters: Even if an attacker steals your password (data breach, phishing), they still can't log in without the physical device running the authenticator app.

OAuth 2.0

What it is: A protocol that lets a user log in to one application using their identity from another (like logging in with your Google account).

Authorization Code Flow (what we use): 1. User clicks "Login with Google" 2. They're redirected to Google to authenticate 3. Google sends back a one-time code to our callback URL 4. Our server exchanges that code for an access token (server-to-server, secret never exposed to browser) 5. We use the token to get the user's email from Google 6. We match that email to our local user database

Why Authorization Code Flow specifically: The access token never touches the user's browser, making it resistant to interception. The client secret stays on our server.

Audit Logging

What it is: An append-only record of everything that happens in the system — who did what, when, and to what.

Why immutable: If someone commits fraud and then tries to delete the logs, the trail would be broken and obvious. In real systems, logs go to a separate, write-only system (like a SIEM — Security Information and Event Management system).

What we log: Every login (success and failure), every role grant and revoke, every approval decision, every SoD violation, every MFA event, every OAuth login.

ISO 27001 Alignment

What it is: An international standard for information security management. It specifies controls companies should have in place.

Relevant controls in our project: - **A.9 — Access Control:** RBAC, least privilege, approval workflows, access reviews — all covered - **A.9.2 — User Access Management:** Joiner/Mover/Leaver lifecycle — directly covered - **A.9.4 — System and Application Access Control:** Authentication, MFA, session management — covered - **A.12.4 — Logging and Monitoring:** Audit log, immutable records, CSV exports — covered

What to say in an interview: "I designed the platform with ISO 27001 Section 9 (Access Control) in mind. Features like time-bound access, recertification cycles, SoD enforcement, and the audit trail all align directly with controls in that standard."

SECTION 7 — REAL-WORLD USE CASES

Use Case 1 — New Employee Joins (Joiner)

Scenario: A new Finance Analyst joins the company on Monday.

Without IAM: IT gets an email, creates an account, guesses which systems they need, the person might have too much access or too little.

With our platform: - Admin opens the Joiner workflow - Enters name, email, department (Finance), manager, and selects the Finance resource role - System checks SoD — Finance + Security would be blocked, but Finance alone is fine - Account is created, Finance role assigned, everything logged under JML_JOINER - Manager is notified, employee can log in from day one with exactly the right access

---

Use Case 2 — Employee Requests Additional Access

Scenario: The same Finance Analyst now needs temporary access to the HR system to run a cross-department report.

Flow: - Employee submits access request with reason: "Need HR data for Q3 Finance-HR headcount reconciliation report" - Reason is 30+ characters — passes justification check - Policy engine checks: Finance + HR = SoD VIOLATION — BLOCKED - Request is rejected automatically with reason: "SoD Conflict: Finance cannot be combined with HR"

This prevents the analyst from ever being in a position to manipulate payroll data.

Use Case 3 — Employee Moves to a New Department (Mover)

Scenario: A Finance Analyst is promoted to IT Security Manager.

Flow: - Admin opens Mover workflow for the user - Revokes Finance role, adds Security role - System checks SoD on the resulting role set - Department updated, manager updated - All changes logged under JML_MOVER - The employee's new access takes effect immediately, old access is removed

---

Use Case 4 — Employee Leaves (Leaver)

Scenario: An IT employee resigns abruptly on a Friday afternoon.

Flow: - Admin opens Leaver workflow - One click: account disabled, IT role revoked, any pending access requests auto-cancelled - Logged under JML_LEAVER - Even if the employee remembered their password, logging in now returns "Account disabled" - The audit log shows the exact time of offboarding

---

Use Case 5 — Quarterly Access Review

Scenario: It's the end of Q3. Security team must certify all access is still appropriate.

Flow: - Admin starts a review cycle named "Q3-2026" - Goes through the list — employee X moved to Marketing 3 months ago but still has Finance access - Decision: Revoke -> Finance role removed -> logged as REVIEW_COMPLETED + ROLE_REVOKED - At the end, the cycle is closed — all remaining roles certified - Full report available for export as CSV for the compliance audit

---

Use Case 6 — Orphan Account Found

Scenario: During the review, the system flags that a user marked Inactive (they left 2 months ago via the HR system) still has an active IT role because someone forgot to run the Leaver workflow.

Flow: - Orphan detection report shows the user - Admin clicks "Remediate" — all roles revoked in one click - Logged as ORPHAN_REMEDIATED - Security risk eliminated

---

SECTION 8 — HOW TO RUN THE PROJECT

Prerequisites

- Python 3.9 or above installed
- pip package manager

Step 1 — Get the code

```
git clone https://github.com/sreekaretta/Enterprise-Identity-Access-Management-IAM-Platform.git
cd Enterprise-Identity-Access-Management-IAM-Platform
```

Step 2 — Install dependencies

```
pip install -r requirements.txt
```

This installs: Flask, Flask-Login, Flask-SQLAlchemy, bcrypt, pyotp, cryptography, requests.

Step 3 — Create and seed the database

```
python seed.py
```

This creates the SQLite database file (`iam.db`) and populates it with: - Demo users (Admin, Manager, 2 Employees, 1 Inactive orphan) - 4 resource roles (HR, Finance, IT, Security) with classifications - 2 SoD rules (Finance+Security, Finance+HR)

Step 4 — (Optional) Enable HTTPS

```
python generate_cert.py
```

This generates `cert.pem` and `key.pem` using a self-signed certificate (valid 365 days). The app automatically uses them if they exist.

Step 5 — Run the app

```
python app.py
```

Open your browser to: `https://127.0.0.1:5000`

If you enabled HTTPS, you'll see a browser security warning. Click **Advanced -> Proceed to 127.0.0.1**. This is expected for self-signed certificates.

Demo Accounts

Email	Password	Role
admin@iam.lab	Password123!	Admin — full access
manager@iam.lab	Password123!	Manager — approvals + team view
employee@iam.lab	Password123!	Employee — request access
ravi@iam.lab	Password123!	Employee — request access
leftco@iam.lab	Password123!	Inactive — orphan account demo

Things to Try

1. **Login as employee** -> Request Access -> submit a Finance request with a reason (30+ chars)
2. **Login as manager** -> approve the request
3. **Login as admin** -> approve the request -> see the role provisioned
4. **Admin -> Users -> employee** -> try to assign Finance + HR simultaneously -> see SoD block
5. **Admin -> JML -> Joiner** -> create a new user with Finance + Security roles -> see SoD block
6. **Admin -> Access Reviews -> Start Cycle** -> name it "Q4-2026" -> certify roles
7. **Admin -> Access Reviews -> Orphans** -> see leftco@iam.lab flagged as orphan
8. **Any user -> Navbar dropdown -> MFA Settings** -> enable TOTP -> scan QR with Google Authenticator
9. **Admin -> Reports -> Audit Log** -> see every action logged
10. **Admin -> SoD Rules** -> add a new SoD rule

SECTION 9 — INTERVIEW PREPARATION

Q1: "Walk me through this project."

Answer: "I built an enterprise-style Identity and Access Management platform from scratch using Python and Flask. The core idea is managing the full lifecycle of user access — from when someone joins a company, through any role changes, to when they leave.

The platform has three system roles — Admin, Manager, and Employee — and four resource roles representing company data areas: HR, Finance, IT, and Security. Access to those resources has to be requested, approved by a manager, then approved by an admin, before it's granted.

I added several security controls on top of that: Segregation of Duties enforcement that blocks role combinations that could enable fraud, quarterly access review cycles, orphan and dormant account detection, time-bound access with expiry dates, and an immutable audit log of everything.

For authentication, I implemented bcrypt password hashing, TOTP two-factor authentication compatible with Google Authenticator, and OAuth 2.0 Authorization Code Flow with Google. I also set up HTTPS using a self-signed TLS certificate.

Everything aligns with ISO 27001 Section 9 access control principles."

Q2: "Why did you build it?"

Answer: "I wanted to build something that demonstrates real enterprise security concepts — not just a CRUD app. IAM is fundamental to every organisation's security posture, and yet it's often the most neglected area.

I also specifically chose this project because I was looking at roles that required knowledge of access control, identity governance, authentication technologies, and compliance — and I wanted to build something that covers all of those in one place."

Q3: "Why did you build it in this particular order?"

Answer: "I followed the natural dependency chain.

First the data models — because everything else depends on knowing what a User, Role, and Assignment look like. Then authentication — because you can't do access control without knowing who's logged in. Then RBAC — because once you know who someone is, you need rules for what they can see. Then JML — because that's how access gets created and removed. Then approvals — because access shouldn't be granted without a check. Then SoD and policy engine — because they're constraints on the provisioning that was already built. Then access reviews, orphan detection — because they're ongoing checks on what's already been granted. Then MFA and OAuth — because they strengthen a login system that was already working. And finally the audit log grew throughout, capturing each feature as it was added."

Q4: "What is RBAC and how did you implement it?"

Answer: "RBAC stands for Role-Based Access Control. Instead of assigning permissions directly to individual users, you define roles and assign permissions to roles. Then users are assigned roles.

In my project I have two layers. The system role (Admin, Manager, Employee) controls what pages and workflows you can access — enforced by a `role_required()` decorator on every route. The resource roles (HR, Finance, IT, Security) represent access to company data systems — stored as UserRole records in the database, with who assigned them, when, and an optional expiry date."

Q5: "What is Segregation of Duties and why does it matter?"

Answer: "Segregation of Duties means no single person should be able to complete a sensitive process entirely on their own — because that creates opportunities for fraud or error.

The classic example: the same person shouldn't be able to create a supplier and also approve payments to that supplier, because they could pay themselves or a fake company.

In my project, I have a SodRule table where admins configure which pairs of roles conflict. The policy engine checks this before every role grant. For example, Finance and HR are blocked together — someone who can process payroll and also manage employee records could add fake employees to pay. Finance and Security are blocked — someone who can access financial systems and also the security systems could cover their tracks.

The check runs in a centralised `check_policy()` function that every role-granting route calls, so no path around it exists."

Q6: "How does the Joiner–Mover–Leaver lifecycle work?"

Answer: "JML is a framework for managing user access throughout their time at a company.

Joiner: when a new employee starts, the admin runs the Joiner workflow — creates the account, assigns starter roles, sets the manager. SoD is checked even on the initial role set.

Mover: when someone changes department, the Mover workflow lets the admin revoke old roles and add new ones in a single transaction. It recalculates the resulting role set and runs SoD check on that.

Leaver: when someone leaves, the Leaver workflow does three things atomically: disables the account, revokes all active roles, and auto-cancels any pending access requests. Everything is logged under JML_LEAVER. Even if the person remembers their password, they cannot log in because the account status is Inactive."

Q7: "What is least privilege?"

Answer: "Least privilege means users should have the minimum access needed to do their job — nothing more.

In practice: a Finance Analyst needs access to Finance data. They don't need HR, IT, or Security access. Giving them more than they need increases the blast radius if their account is compromised — an attacker who takes over that account gets everything the user had.

In my project: users start with no resource roles. They must actively request any additional access, provide a business reason, get it approved by their manager and an admin, and pass the policy check before it's granted."

Q8: "How do access reviews work?"

Answer: "Access reviews are periodic campaigns where someone — usually an admin or a manager — goes through a list of all active access assignments and certifies: does this person still need this role?

In my project, the admin starts a named review cycle, like 'Q3-2026'. They see a table of every active UserRole in the system — user name, role name, when it was assigned. For each row they click Keep or Revoke. Revoke immediately sets the UserRole status to Revoked and writes a log entry. At the end they can close the cycle, which auto-certifies any remaining unreviewed assignments as Keep.

This is important because access tends to accumulate. Someone gets temporary access for a project, the project ends, but nobody removes the access. Reviews catch that."

Q9: "How did you implement MFA and OAuth 2.0?"

Answer: "For MFA, I used the pyotp library which implements the TOTP standard (RFC 6238). During setup, we generate a random base32 secret and display it as a QR code using the qrcodejs library in the browser. The user scans it with Google Authenticator. After that, at login, after the password is accepted, if the user has TOTP enabled, we redirect them to a code entry page instead of logging them in. We verify the code using `pyotp.TOTP(secret).verify(code, valid_window=1)` — the valid_window allows a 30-second tolerance for clock drift.

For OAuth 2.0, I implemented the Authorization Code Flow with Google as the Identity Provider. The key steps are: generate a random state token for CSRF protection, redirect to Google with that state, verify the state when Google redirects back (CSRF check), exchange the authorization code for an access token server-to-server (the client secret never touches the browser), use the access token to get the user's email from Google's userinfo endpoint, and match that email to a local account. If the user also has TOTP enabled, they're still required to enter their authenticator code even when logging in via OAuth."

Q10: "How does this align with ISO 27001?"

Answer: "ISO 27001 Annex A Section 9 covers Access Control. Our platform directly implements those controls:

A.9.1 — Business requirements of access control: we have a written justification requirement and an approval workflow before access is granted.

A.9.2 — User access management: the JML lifecycle handles provisioning (Joiner), changes (Mover), and deprovisioning (Leaver). Access review cycles handle recertification.

A.9.3 — User responsibilities: users must justify their requests, MFA strengthens authentication.

A.9.4 — System and application access control: RBAC, least privilege, SoD enforcement, session management.

A.12.4 — Logging and monitoring: our immutable audit log covers all significant events and is exportable for evidence in an audit."

Q11: "What are its limitations?"

Answer (be honest): "A few honest limitations:

The database is SQLite — it's file-based and not suitable for production scale. In a real deployment you'd use PostgreSQL or MySQL.

The HTTPS certificate is self-signed, which browsers warn about. In production you'd use a certificate from a trusted Certificate Authority like Let's Encrypt.

OAuth is set up for Google only. In an enterprise, you'd connect to Okta, Azure AD, or a corporate SAML Identity Provider.

The SoD rules are configured manually by an admin. Enterprise IAM tools like SailPoint or Saviynt have role mining features that suggest SoD rules automatically based on existing access patterns.

The access reviews are manual. In production, review tasks would be automatically assigned to managers with email notifications and deadlines.

There's no provisioning to external systems — in a real tool, approving a request would automatically create the account in Active Directory, Salesforce, or whatever the target system is."

Q12: "How would you scale or improve it?"

Answer: "Several ways:

Replace SQLite with PostgreSQL for concurrent access and better performance.

Add automatic provisioning to external systems via SCIM (System for Cross-domain Identity Management) protocol — when an access request is approved, the platform automatically creates the account in the target system.

Integrate with an enterprise Identity Provider like Azure AD or Okta via SAML 2.0 for true SSO.

Add automated access review assignments — the system sends review tasks to managers by email with a deadline.

Add risk scoring — flag users who have many high-classification roles, or whose access patterns deviate from their peer group.

Build a REST API so other internal tools can query and manage access programmatically.

Add multi-tenancy support so it can manage identity for multiple departments or subsidiary companies."

Q13: "How does this compare to real IAM tools like Saviynt or Okta?"

Answer: "Saviynt and Okta are enterprise-grade products with teams of hundreds working on them for years, so the comparison is really about demonstrating understanding of what they do rather than claiming equivalence.

Okta is primarily focused on authentication — SSO, MFA, and federation across many applications. Our project demonstrates the same OAuth 2.0 and TOTP concepts that sit at Okta's core.

Saviynt is more of an Identity Governance and Administration (IGA) tool — it handles access requests, certifications, SoD, role management, and provisioning to downstream systems. Our project covers all of those concepts: JML lifecycle, approval workflows, SoD enforcement, access reviews, and orphan detection.

The key difference is that production tools like Saviynt connect to hundreds of real systems (SAP, Salesforce, Active Directory) and have AI-driven role mining and risk scoring. Our project demonstrates the same governance concepts and would serve as an excellent foundation for understanding how those tools work and why they exist."

SECTION 10 — QUICK REVISION CHEAT-SHEET

The Project in One Line

An enterprise IAM platform that controls user access through RBAC, JML lifecycle, SoD enforcement, MFA, and OAuth 2.0 — with full audit logging aligned to ISO 27001.

The Flow (in 10 steps)

```
1. User created (Joiner)
2. Access requested (with 30+ char justification)
3. Manager approves
4. Admin approves
5. Policy engine runs (SoD check + classification check)
6. Role provisioned (UserRole created, with optional expiry)
7. Periodic review (Keep or Revoke each active role)
8. Orphan/Dormant detection (ongoing hygiene checks)
9. Deprovisioning (Leaver / review / expiry)
10. Audit log captures everything
```

Key Models

Model	Purpose
User	Person — name, email, dept, system role, status, MFA fields
Role	Resource — name, classification (Restricted/Confidential/Internal/Public)
UserRole	Link between User and Role — who assigned it, when, expiry, status
AccessRequest	Approval workflow — stage, decisions, timestamps
SodRule	A pair of roles that cannot coexist
AuditLog	Immutable record of every action
AccessReview	Decision in a recertification cycle

SoD Rules (seeded)

- Finance + Security = blocked (fraud concealment)
- Finance + HR = blocked (payroll manipulation)

Authentication Stack

Technology	Library	What it does
Password hashing	bcrypt	Hashes passwords before storing — never plain text
Session management	Flask-Login	Tracks who is logged in
TOTP/MFA	pyotp	6-digit code every 30 seconds — Google Authenticator
OAuth 2.0	requests + urllib	Login with Google via Authorization Code Flow
HTTPS/TLS	cryptography	Self-signed certificate, auto-loaded at startup

---

ISO 27001 Map

Our Feature	ISO 27001 Control
RBAC + Least Privilege	A.9.1, A.9.4
JML Lifecycle	A.9.2.1, A.9.2.6
Approval Workflow	A.9.2.2
Access Reviews	A.9.2.5
SoD	A.6.1.2
Audit Log	A.12.4
MFA	A.9.4.2
HTTPS	A.10.1

The Build Order (why each piece comes when it does)

```
1. Models          -> foundation everything else rests on
2. Auth (login)    -> must know WHO before controlling WHAT
3. RBAC            -> access control after identity is established
4. JML             -> lifecycle management on top of users+roles
5. Approvals       -> controlled way to grant access
6. SoD + Policy    -> constraints wrapping the provisioning
7. Classification -> refinement of the access request process
8. Reviews         -> ongoing check on already-granted access
9. Orphan/Dormant -> detective controls catching what slipped through
10. MFA + OAuth    -> strengthen authentication on top of working login
11. Audit log      -> grew with every feature, wraps everything
```

Key Interview Phrases

- "Centralised policy engine — single point of enforcement, no way around it"
- "Atomic deprovisioning — account, roles, and requests all handled in one transaction"
- "State machine approval workflow — requests move through defined stages in order"
- "Immutable audit log — append-only, every action captured with actor and timestamp"
- "Bidirectional SoD check — Finance+Security is the same conflict as Security+Finance"
- "valid_window=1 on TOTP — allows ±30 seconds for clock drift between server and phone"
- "Authorization Code Flow — access token stays server-side, never touches the browser"
- "ISO 27001 Section 9 — access control, least privilege, recertification, audit trail"

End of Document — Enterprise IAM Platform Study Guide